Evidence review: 15-position prime-norm scalar radix

Question. Does the `268−τ` prime-norm radix provide a verified 15-position integer recoding, and what does the existing paired panel establish against the 16-position `190−τ` format?

Scope and method. This report audits the frozen artifacts in this directory; it does not run a new benchmark. The search enumerated the declared `T=240…320` prime-norm candidates, checked selector/radius/terminal bounds, compared the six-candidate selector with the bounded global search on all sign-canonical residues, and replayed the selected recoding. Integer checks are complemented by independent affine point replay and native full-residue/holdout tests. The performance comparison uses panel `2481bd3c0f7e491698694ccb039f1048` on RunPod instance `thyvmdqmzsz5ik`, comparing radix 15 (`N=71,023`) against radix 16 (`N=35,533`) on the same 128 frozen scalar inputs. No secp256k1 curve identity or subgroup claim is needed for the scalar-format comparison; the independent group replay is specifically over secp256k1.

Findings

- `N(268−τ)=71,023` is prime and the exact declared sweep has twelve prime-norm candidates. **Proved and independently checked.** `NearestOrbit71023Bound.lean` contains `norm_sqrt_cap`, `norm_trial_division`, and `norm_prime : Nat.Prime 71023`, checked by Lean's kernel; `frontier15.py` and `frontier15.json` record the search. The sweep establishes only the stated `T=240…320` range.
- The selected integer recoding has 15 steps and terminal norm below 82^2. **Proved for the formalized integer recoding.** Selector, finite digit certificate, contraction, and trajectory declarations in the six Lean files linked by `README.md`; checked logs are retained. The candidate-specific secp256k1 group-action theorem remains unformalized.
- The 15-position path matches independent point multiplication on the checked samples. **Independently verified on bounded samples.** 388 affine replays and a 128-scalar native holdout, documented in `group-replay268.json`, `screen268.json`, and the native test/build logs. This is sample-based group validation, not a proof of the curve-specific bridge for all inputs.
- Median sustained hot-table throughput was 1.024327× radix 16, with repeat-bootstrap 95% interval `[1.020012, 1.031215]`. **Measured; the panel passes its A/A and interval gate, with a repository evidence requirement still missing.** `tier1-analysis.json`, `tier1-rules.json`, `tier1-manifest.json`, and the 96 raw arms / 48 pair receipts under `2481bd3c0f7e491698694ccb039f1048/` retain the panel. The raw arm receipts omit per-process peak RSS, required by the base-branch hardware-evidence rules. The available cgroup peak is container-wide and not attributable to either process. The result is for repeated warm-table work on this allocation and frozen scalar law; it does not establish cold-start or end-to-end library speedup.
- The candidate uses less setup or memory. **Refuted by recorded measurements.** Mean preparation was 2,114.12 ms versus 1,257.82 ms and retained table memory was 12,336,756 versus 6,829,948 bytes (candidate/reference). These costs are outside the online timing interval and are shown separately.

The generic runner's archived `summary.json` is retained unchanged. Its `paired_speedup` value (1.003080) is the median from the pre-comparison A/A calibration case, and its generic `speedup_claim_eligible=false` describes that runner-level aggregate. The radix comparison result is separately derived by `analyze_tier1.py` from the `compare` case after checking both adjacent A/A calibration cases; the comparison median and interval are in `tier1-analysis.json`. The two summaries describe different calculations and must not be conflated.

The 96 raw arm receipts record child user/system CPU time, wall/online time, cgroup throttling and memory-event deltas, steal time, faults, and context switches. The audit found zero throttling, steal, major faults, or OOM events; involuntary context switches total 647 across all arms. The receipts do not retain the runner's per-child peak RSS field. The preflight records a 1,781,030,912-byte cgroup peak, but that is container-wide and cannot stand in for per-process peak RSS. Thus the frozen panel passes the comparison's service gate, while its retained data do not satisfy the repository's complete per-run memory-evidence requirement. Reconstructing that missing process statistic from the other counters is not valid.

The panel manifest records hashes for both benchmark binaries and source files. However, the candidate overlay requires a parent source snapshot whose hash (`e195374c…`) does not match the source file committed in the base tree; the manifest's external `/workspace/bench-sources/…` snapshots are not present in the run archive or this review workspace. The PR preserves the overlay procedure and hashes, but that historical native build cannot be reconstructed from the checked-out evidence alone. The input fixture itself is preserved in the base experiment tree and its SHA-256 matches the panel manifest.

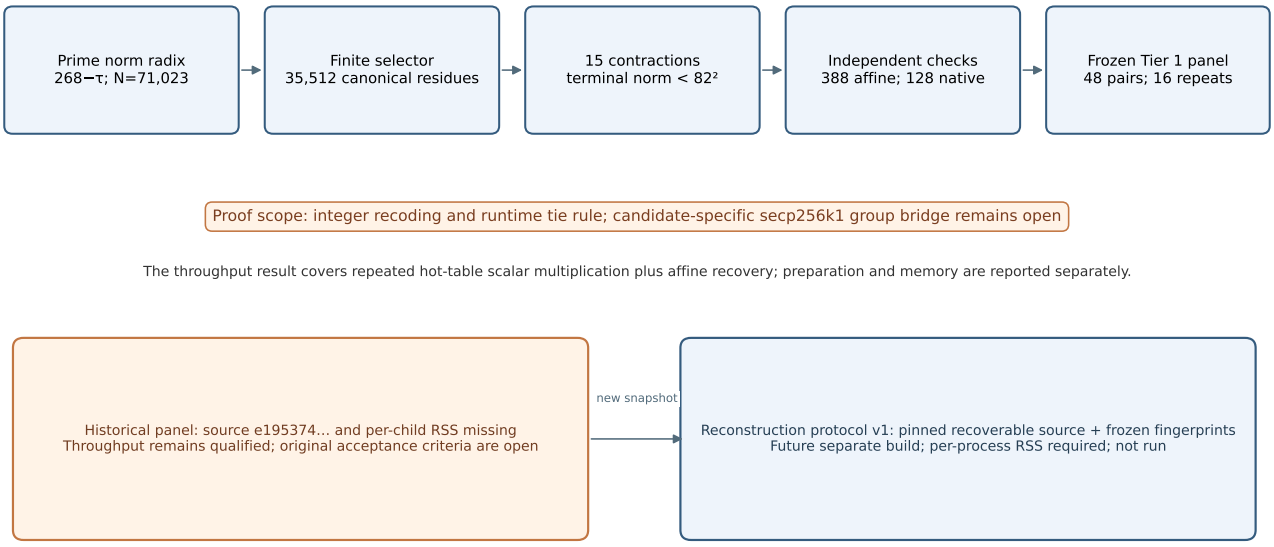
Versioned native reconstruction path

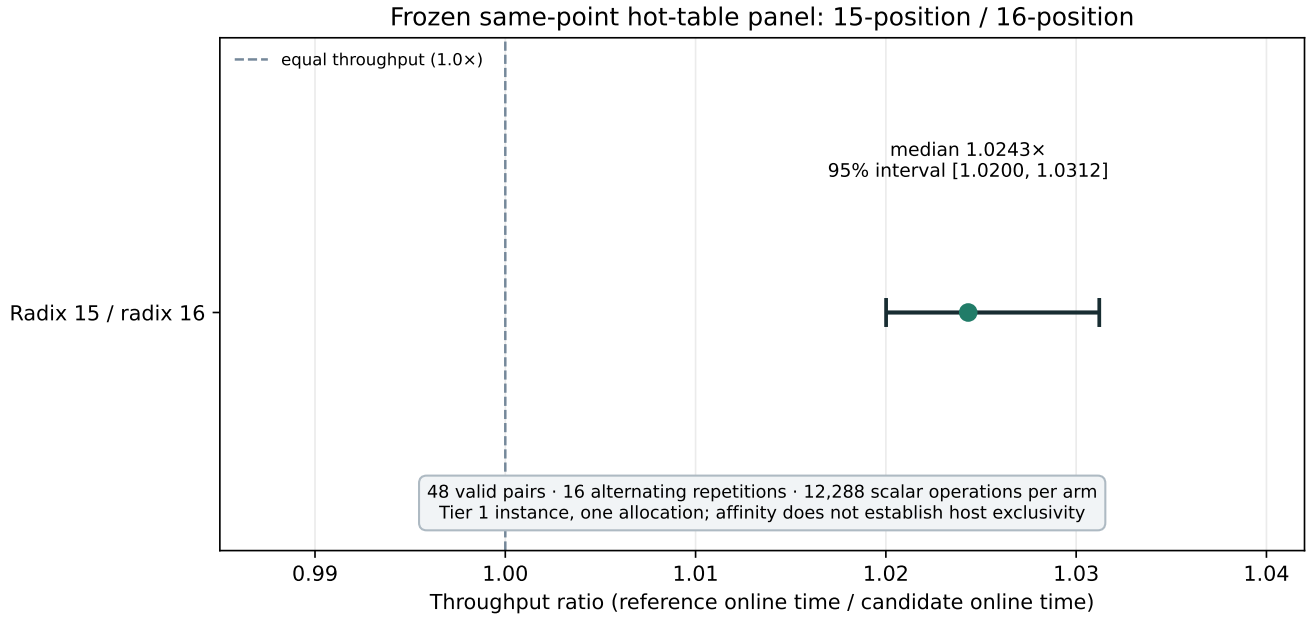
I repeated the local source search and confirmed that no file matches historical parent SHA-256 `e195374…`. The recoverable source at base commit `37a57501…` is a different snapshot: `unit_orbit_windows.rs` hashes to `7a6492af…` and lacks the split-prime section required by `native268_overlay.py`. I therefore did not run the historical overlay or claim to reconstruct its executable. Exact fingerprints are in the versioned protocol.

`NATIVE_RECONSTRUCTION_PROTOCOL-v1.md` records a separate future path based on that pinned, recoverable Rust source, with Cargo/source hashes, a versioned port requirement, and pre-timing gates. The frozen input is the byte-identical `selective-holdout.json` with SHA-256 `7f4672b7…`. The protocol pins the existing frozen-output fingerprints (`group-replay268.json`, `screen268.json`, and `linux-preflight.stdout`) and requires per-child RSS with process identity and units in any new run. It has not been executed and does not repair the historical panel's source or RSS gap. The named open blocker and acceptance criteria are tracked in `problems/prime-j0-radix15-native-provenance.md`.

Visuals and graph-set audit

`figures.py` is the editable source for the evidence-flow diagram and the quantitative throughput graph. Their SVG and PDF renders are alongside the source; `REPORT-2026-10-19.pdf` contains this report and both visuals. The throughput figure uses the existing paired median and repeat-bootstrap interval and labels the sample, timing scope, tier, and host-isolation limit.





I checked the existing `experiments/ec-arithmetic-optimizations/` result figures and the `experiments/volcanoic/figures/` graph set. The former compares arithmetic-layer workloads and has no matching radix-15/radix-16 observation to update; the latter tracks isogeny/IC surface structure, which this scalar-format result does not change. Neither canonical graph's source data or conclusion changes here. The local graph added above is the quantitative visualization for this newly measured, separate workload. No curve registry, IC route, benchmark baseline, or dashboard row is changed because this is not an IC candidate or production baseline measurement.

The historical throughput plot remains current: this review added no timing observation or changed metric. The evidence-flow diagram now distinguishes the historical panel's missing source/RSS provenance from the pinned-source reconstruction protocol, which is a future path rather than a result.

Limitations and open checks

The paired interval describes repetition variation on one cloud instance, not host-exclusive performance or variability across providers, CPUs, or scalar distributions. CPU affinity records process placement only. One-call latency was not measured. Table construction, preparation, and memory favor the 16-position reference. The formal proof covers the integer recoding; the curve-specific group bridge is supported by independent bounded replays but remains a formalization gap. These results do not establish production integration or end-to-end secp256k1 library speedup.

The exact source for the measured binaries is bound by the recorded source hashes and native overlay receipts; the raw panel archive contains the run receipts, while the PR preserves the overlay and source-generation scripts. Reproducing the historical remote allocation is not part of this review and no new performance run was performed.

Reproduction

Run `python3 figures.py` to regenerate the editable SVG/PDF visuals from `tier1-analysis.json` and `tier1-rules.json`. Run `python3 check_frozen.py` from the repository root to replay the frontier search, integer screen, group checks, and tier1 audit and compare their outputs against the preserved JSON; all four matched in this review. The new CI workflow runs this checker and builds the pinned Lean modules using `check_lean268.sh`. Lean is unavailable in this review environment, so I could not run the proof script locally; preexisting proof logs audit the earlier declarations, and the CI job rechecks the added explicit primality theorem and all modules. The candidate-specific point-group bridge is explicitly pending. The new native reconstruction protocol is future work and was not run.